

Chapter 24 — OOP &



JavaScript Classes

JavaScript Master Study

Documentation • ECMAScript 2026
Standard

50 Core Modules

OOP 4 Pillars

Private Fields (#)

Prototypes & Extends

Practice Lab

Chapter 24 Learning Goals & Architecture Roadmap

OOP (Object-Oriented Programming) হলো আধুনিক সফটওয়্যার ইঞ্জিনিয়ারিংয়ের মূল ভিত্তি। বিশেষ করে **React, Node.js, Express, NestJS, এবং large-scale enterprise application** ডেভেলপমেন্টে পরিষ্কার ডেটা মডেলিং এবং মডুলার কোড স্ট্রাকচার সাজাতে ক্লাস এবং অবজেক্টের কনসেপ্ট অপরিহার্য। এই চ্যাপ্টারে আমরা একদম গ্রাউন্ড লেভেল থেকে ইন্টারনাল প্রোটোটাইপ মেকানিজম পর্যন্ত নিখুঁতভাবে শিখব।

- ✓ OOP কী এবং কেন দরকার
- ✓ Object-oriented programming-এর 4 pillars
- ✓ Class কী
- ✓ Object কী
- ✓ Constructor
- ✓ this
- ✓ Instance properties
- ✓ Instance methods
- ✓ Static properties/methods
- ✓ Getters & Setters
- ✓ extends
- ✓ super()
- ✓ Method overriding
- ✓ Inheritance
- ✓ Encapsulation
- ✓ Private fields #
- ✓ Polymorphism
- ✓ Abstraction
- ✓ Composition
- ✓ instanceof
- ✓ Class vs Constructor Function
- ✓ Prototype-এর সাথে Class-এর সম্পর্ক
- ✓ Real-world project architecture
- ✓ Common mistakes
- ✓ Interview questions

Part 01 — OOP Fundamentals, Objects & Class Syntax (24.1 – 24.10)

24.1 — OOP কী?

Module 24.1

OOP = Object-Oriented Programming এটি এমন একটি programming approach যেখানে program-এর data এবং সেই data-এর সাথে কাজ করার functions-কে একসাথে **object** হিসেবে organize করা হয়। সহজভাবে: > **Data + Behavior = Object**

◆ Real-Life Example

ধরো একটি **Student**। Student-এর data:

► Console Output

```
name
id
department
cgpa
```

Student-এর behavior:

► Console Output

```
study()
attendClass()
takeExam()
```

JavaScript-এ আমরা এগুলো একটি object/class-এর মাধ্যমে organize করতে পারি। ---

🚀 24.2 — OOP কেন দরকার?

Module 24.2

ছোট program-এ সাধারণ function এবং object দিয়েই কাজ করা যায়। কিন্তু বড় application হলে সমস্যা হয়। ধরো university system:

► Console Output

```
Student
Teacher
Admin
Course
Department
Attendance
Result
Payment
```

প্রত্যেকটির আলাদা data এবং behavior আছে। OOP ব্যবহার করলে code:

- organized হয়
- reusable হয়
- maintain করা সহজ হয়
- বড় application manage করা সহজ হয়
- inheritance ব্যবহার করা যায়
- encapsulation করা যায়

🚀 24.3 — OOP-এর 4 Pillars

Module 24.3

OOP-এর চারটি মূল concept: | Pillar | সহজ অর্থ | | ----- | -----
--- | | Encapsulation | Data + Methods একসাথে রাখা এবং access control করা | | Abstraction | প্রয়োজনীয় অংশ দেখানো, internal complexity hide করা | | Inheritance | একটি class থেকে অন্য class-এর বৈশিষ্ট্য নেওয়া | | Polymorphism | একই method/interface বিভিন্নভাবে কাজ করা | মনে রাখবে: > **E-A-I-P** Encapsulation → Abstraction → Inheritance → Polymorphism ---

📌 24.4 — Object কী?

Module 24.4

Object হলো key-value pair-এর collection।

JavaScript

JAVASCRIPT

```
const student = {
  name: "Shariar",
  age: 23,
  department: "CSE",

  study() {
    console.log("Student is studying");
  }
};
```

এখানে:

▶ Console Output

```
name
age
department
```

হলো data আর:

JavaScript

JAVASCRIPT

```
study()
```

হলো behavior। ---

Class হলো object তৈরির একটি **blueprint/template**।

♦ Real-Life Example

একটি বাড়ির blueprint ধরো:

► Console Output

```
Bedrooms  
Bathrooms  
Kitchen  
Garage
```

Blueprint নিজে বাড়ি নয়। Blueprint থেকে অনেক বাড়ি বানানো যায়। ঠিক একইভাবে:

JavaScript

JAVASCRIPT

```
class Student {  
  
}
```

হলো blueprint। আর:

JavaScript

JAVASCRIPT

```
const student1 = new Student();  
const student2 = new Student();
```

হলো সেই blueprint থেকে তৈরি objects। ---

JavaScript

JAVASCRIPT

```
class Student {  
  
}
```

Object তৈরি:

JavaScript

JAVASCRIPT

```
const student1 = new Student();
```

এখানে:

📖 ASCII Concept Architecture

```
class → blueprint  
Student → class name  
new → নতুন object তৈরি  
student1 → object/instance
```

Class-এর ভিতরে `constructor()` একটি special method। Object তৈরি হওয়ার সময় এটি automatically execute হয়।

JavaScript

JAVASCRIPT

```
class Student {  
  
  constructor() {  
    console.log("Student object created");  
  }  
  
}  
  
const student1 = new Student();
```

▶ Console Output

```
Student object created
```

কারণ:

JavaScript

JAVASCRIPT

```
new Student();
```

চললে constructor automatically call হয়। ---

এখন student-এর information পাঠাই।

JavaScript

JAVASCRIPT

```
class Student {  
  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
}
```

Object:

JavaScript

JAVASCRIPT

```
const student1 = new Student("Shariar", 23);  
  
console.log(student1.name);  
console.log(student1.age);
```

▶ Console Output

```
Shariar  
23
```

OOP-এ `this` অত্যন্ত গুরুত্বপূর্ণ।

JavaScript

JAVASCRIPT

```
class Student {

  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

}
```

এখানে:

JavaScript

JAVASCRIPT

```
this.name
```

মানে: > যে object তৈরি হচ্ছে, সেই object-এর `name` property। যখন:

JavaScript

JAVASCRIPT

```
const student1 = new Student("Shariar", 23);
```

তখন:

JavaScript

JAVASCRIPT

```
this
```

মূলত `student1` object-কে refer করছে। --- # 🧠 সহজভাবে

JavaScript

JAVASCRIPT

```
this.name = name;
```

মানে:

▶ Console Output

```
object-এর name = parameter-এর name
```

একটি class থেকে অনেক object তৈরি করা যায়।

JavaScript

JAVASCRIPT

```
class Student {  
  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
}  
  
const student1 = new Student("Shariar", 23);  
const student2 = new Student("Rahim", 22);  
const student3 = new Student("Karim", 24);  
  
console.log(student1);  
console.log(student2);  
console.log(student3);
```

প্রতিটি object-এর নিজস্ব data থাকবে। ---

📌 24.11 — Instance Method

Module 24.11

Class-এর ভিতরে function লিখলে সেটাকে সাধারণত **method** বলা হয়।

 JavaScript

JAVASCRIPT

```
class Student {  
  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  introduce() {  
    console.log(`My name is ${this.name}`);  
  }  
  
}
```

ব্যবহার:

 JavaScript

JAVASCRIPT

```
const student = new Student("Shariar", 23);  
  
student.introduce();
```

▶ Console Output

My name is Shariar

JavaScript

JAVASCRIPT

```
class Student {  
  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  introduce() {  
    console.log(`My name is ${this.name}`);  
  }  
  
  study() {  
    console.log(`${this.name} is studying`);  
  }  
  
  sleep() {  
    console.log(`${this.name} is sleeping`);  
  }  
  
}  
  
const student = new Student("Shariar", 23);  
  
student.introduce();  
student.study();  
student.sleep();
```

▶ Console Output

```
My name is Shariar  
Shariar is studying  
Shariar is sleeping
```

JavaScript

JAVASCRIPT

```
class BankAccount {

  constructor(owner, balance) {
    this.owner = owner;
    this.balance = balance;
  }

  deposit(amount) {
    this.balance += amount;
  }

  withdraw(amount) {
    if (amount > this.balance) {
      console.log("Insufficient balance");
      return;
    }

    this.balance -= amount;
  }

  showBalance() {
    console.log(`Balance: ${this.balance}`);
  }
}

const account = new BankAccount("Shariar", 10000);

account.deposit(5000);
account.withdraw(2000);
account.showBalance();
```

► Console Output

Balance: 13000

কারণ:

► Console Output

10000 + 5000 - 2000
= 13000

যে properties প্রতিটি object-এর জন্য আলাদা হয় সেগুলো instance properties।

 JavaScript

JAVASCRIPT

```
class User {  
  
  constructor(name, email) {  
    this.name = name;  
    this.email = email;  
  }  
  
}
```

 JavaScript

JAVASCRIPT

```
const user1 = new User("Shariar", "shariar@example.com");  
const user2 = new User("Rahim", "rahim@example.com");
```

এখানে:

 ASCII Concept Architecture

```
user1.name → Shariar  
user2.name → Rahim
```

Static property class-এর সাথে associated থাকে, individual object-এর সাথে নয়।

JavaScript

JAVASCRIPT

```
class User {  
  
    static role = "Student";  
  
}
```

Access:

JavaScript

JAVASCRIPT

```
console.log(User.role);
```

▶ Console Output

Student

কিন্তু:

JavaScript

JAVASCRIPT

```
const user = new User();  
  
console.log(user.role);
```

এটি `undefined` হবে কারণ `role` object-এর property নয়। ---

Static method object দিয়ে নয়, class দিয়ে call করা হয়।

JavaScript

JAVASCRIPT

```
class MathHelper {  
  
    static add(a, b) {  
        return a + b;  
    }  
  
}  
  
console.log(MathHelper.add(10, 20));
```

▶ Console Output

30

এটা:

JavaScript

JAVASCRIPT

MathHelper.add()

দিয়ে call করতে হবে। ---

JavaScript

JAVASCRIPT

```
class User {  
  
  constructor(name) {  
    this.name = name;  
  }  
  
  static createGuest() {  
    return new User("Guest");  
  }  
  
}  
  
const guest = User.createGuest();  
  
console.log(guest.name);
```

▶ Console Output

Guest

এখানে `createGuest()` একটি utility/factory-like static method। ---

Getter দিয়ে property-এর মতো syntax ব্যবহার করে method-এর result পাওয়া যায়।

JavaScript

JAVASCRIPT

```
class Rectangle {  
  
  constructor(width, height) {  
    this.width = width;  
    this.height = height;  
  }  
  
  get area() {  
    return this.width * this.height;  
  }  
}  
  
const rectangle = new Rectangle(10, 5);  
  
console.log(rectangle.area);
```

▶ Console Output

50

খেয়াল করো:

JavaScript

JAVASCRIPT

```
rectangle.area
```

এখানে `()` নেই। কারণ এটি getter! ---

Setter দিয়ে property set করার সময় custom logic চালানো যায়।

JavaScript

JAVASCRIPT

```
class User {  
  
  constructor(name) {  
    this.name = name;  
  }  
  
  set username(value) {  
    this.name = value.trim();  
  }  
  
}  
  
const user = new User("Shariar");  
  
user.username = "  Ripon  ";  
  
console.log(user.name);
```

▶ Console Output

Ripon

 JavaScript

JAVASCRIPT

```
class Person {  
  
  constructor(name) {  
    this._name = name;  
  }  
  
  get name() {  
    return this._name;  
  }  
  
  set name(value) {  
  
    if (value.length < 3) {  
      throw new Error("Name is too short");  
    }  
  
    this._name = value;  
  }  
}  
  
const person = new Person("Shariar");  
  
console.log(person.name);  
  
person.name = "Ripon";  
  
console.log(person.name);
```

Encapsulation মানে data এবং behavior একসাথে রাখা এবং data access/control করা। ধরো Bank Account:

 JavaScript

JAVASCRIPT

```
class BankAccount {  
  
    constructor(balance) {  
        this.balance = balance;  
    }  
  
}
```

এখানে বাইরে থেকে সরাসরি:

 JavaScript

JAVASCRIPT

```
account.balance = -500000;
```

করা সম্ভব। এটি undesirable হতে পারে। Private field ব্যবহার করা যায়। ---

Modern JavaScript-৭:

JavaScript

JAVASCRIPT

```

class BankAccount {

    #balance;

    constructor(balance) {
        this.#balance = balance;
    }

    getBalance() {
        return this.#balance;
    }

}

const account = new BankAccount(10000);

console.log(account.getBalance());
    
```

▶ Console Output

10000

কিন্তু:

JavaScript

JAVASCRIPT

```

console.log(account.#balance);
    
```

এটি **SyntaxError** হবে। কারণ `#balance` private। ---

📌 24.23 — Private Method

Module 24.23

Methods-ও private করা যায়।

JavaScript

JAVASCRIPT

```
class BankAccount {  
  
    #balance = 0;  
  
    #validateAmount(amount) {  
  
        return amount > 0;  
  
    }  
  
    deposit(amount) {  
  
        if (!this.#validateAmount(amount)) {  
            throw new Error("Invalid amount");  
        }  
  
        this.#balance += amount;  
    }  
  
    getBalance() {  
        return this.#balance;  
    }  
  
}
```

এটি encapsulation-এর ভালো example। ---

Part 03 — Inheritance, Polymorphism, Abstraction & Composition (24.24 – 24.33)

📌 24.24 — Inheritance

Module 24.24

Inheritance মানে একটি class অন্য class-এর properties এবং methods ব্যবহার করতে পারবে। ধরো:

📄 ASCII Concept Architecture

```
Animal  
  ↓  
Dog
```

Dog হলো Animal-এর একটি type। ---

JavaScript

JAVASCRIPT

```
class Animal {  
    eat() {  
        console.log("Animal is eating");  
    }  
}  
  
class Dog extends Animal {  
    bark() {  
        console.log("Dog is barking");  
    }  
}  
  
const dog = new Dog();  
  
dog.eat();  
dog.bark();
```

▶ Console Output

```
Animal is eating  
Dog is barking
```

Dog নিজের method:

▶ Console Output

```
bark()
```

পেয়েছে। Animal থেকে পেয়েছে:

▶ Console Output

```
eat()
```

Child class-এর constructor থাকলে parent constructor call করতে `super()` ব্যবহার করতে হয়।

JavaScript

JAVASCRIPT

```
class Animal {  
  
  constructor(name) {  
    this.name = name;  
  }  
  
}  
  
class Dog extends Animal {  
  
  constructor(name, breed) {  
    super(name);  
    this.breed = breed;  
  }  
  
}  
  
const dog = new Dog("Tommy", "German Shepherd");  
  
console.log(dog.name);  
console.log(dog.breed);
```

▶ Console Output

```
Tommy  
German Shepherd
```

Parent class-এর method call করতেও `super` ব্যবহার করা যায়।

JavaScript

JAVASCRIPT

```
class Animal {  
    speak() {  
        console.log("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    speak() {  
        super.speak();  
  
        console.log("Dog barks");  
    }  
}  
  
const dog = new Dog();  
  
dog.speak();
```

► Console Output

```
Animal makes a sound  
Dog barks
```

Child class parent-এর method নিজের মতো করে redefine করতে পারে।

JavaScript

JAVASCRIPT

```
class Animal {  
  speak() {  
    console.log("Animal sound");  
  }  
}  
  
class Dog extends Animal {  
  speak() {  
    console.log("Woof!");  
  }  
}  
  
const dog = new Dog();  
  
dog.speak();
```

▶ Console Output

Woof!

এটাই method overriding! ---

Poly = many Morph = forms অর্থাৎ: > একই interface/method বিভিন্ন object-এ বিভিন্ন behavior দিতে পারে।

JavaScript

JAVASCRIPT

```
class Animal {  
    speak() {  
        console.log("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    speak() {  
        console.log("Woof!");  
    }  
}  
  
class Cat extends Animal {  
    speak() {  
        console.log("Meow!");  
    }  
}  
  
const animals = [  
    new Dog(),  
    new Cat()  
];  
  
animals.forEach(animal => {  
    animal.speak();  
});
```

▶ Console Output

Woof!
Meow!

একই:

JavaScript

JAVASCRIPT

```
animal.speak()
```

কিন্তু result আলাদা। এটাই polymorphism-এর basic example! ---

কোন object কোনো class-এর instance কিনা check করতে:

JavaScript

JAVASCRIPT

```
class Student {}  
  
const student = new Student();  
  
console.log(student instanceof Student);
```

▶ Console Output

true

আর:

JavaScript

JAVASCRIPT

```
console.log(student instanceof Array);
```

Output:

▶ Console Output

false

একটি realistic example:

JavaScript

JAVASCRIPT

```
class BankAccount {

    #balance;

    constructor(owner, initialBalance = 0) {
        this.owner = owner;
        this.#balance = initialBalance;
    }

    deposit(amount) {

        if (amount ≤ 0) {
            throw new Error("Invalid deposit amount");
        }

        this.#balance += amount;
    }

    withdraw(amount) {

        if (amount > this.#balance) {
            throw new Error("Insufficient balance");
        }

        this.#balance -= amount;
    }

    getBalance() {
        return this.#balance;
    }
}

const account = new BankAccount("Shariar", 10000);

account.deposit(5000);
account.withdraw(2000);

console.log(account.getBalance());
```

Output:

▶ Console Output

13000

এখানে একসাথে ব্যবহার হয়েছে:

- Class

- Constructor
- `this`
- Private field
- Methods
- Encapsulation
- Error handling

Abstraction মানে: > জটিল internal implementation hide করে user-কে প্রয়োজনীয় interface দেওয়া।

♦ Real-Life Example

তুমি ATM ব্যবহার করো:

📄 ASCII Concept Architecture

Insert Card

↓

Enter PIN

↓

Withdraw

তুমি জানো না ATM-এর ভিতরে:

▶ Console Output

Database

Bank Server

Encryption

Transaction System

Network

কীভাবে কাজ করছে। এটাই abstraction-এর ধারণা। JavaScript example:

```
class ATM {

  withdraw(amount) {

    this.#verifyAccount();
    this.#checkBalance();
    this.#processTransaction();

    console.log(`Withdrawn: ${amount}`);
  }

  #verifyAccount() {
    console.log("Account verified");
  }

  #checkBalance() {
    console.log("Balance checked");
  }

  #processTransaction() {
    console.log("Transaction processed");
  }

}

const atm = new ATM();

atm.withdraw(5000);
```

User সঞ্চয়:

```
atm.withdraw(5000);
```

ব্যবহার করছে। Internal process hidden। ---

সবসময় inheritance ব্যবহার করতে হবে এমন নয়। Composition মানে ছোট ছোট object/component combine করে বড় behavior তৈরি করা। ধরো:

JavaScript

JAVASCRIPT

```
const engine = {
  start() {
    console.log("Engine started");
  }
};

const musicSystem = {
  playMusic() {
    console.log("Music playing");
  }
};

const car = {
  engine,
  musicSystem
};

car.engine.start();
car.musicSystem.playMusic();
```

এখানে car-এর:

▶ Console Output

```
Engine
Music System
```

আলাদা components। --- # 🧠 Inheritance बनाम Composition | Inheritance | Composition | | -----
----- | ----- | | IS-A relationship | HAS-A relationship | | Dog IS-A Animal | Car HAS-A
Engine | | `extends` | objects/components combine | | Tight coupling হতে পারে | বেশি flexible হতে পারে |

♦ Example

▶ Console Output

```
Dog IS-A Animal
```

Inheritance। কিন্তু:

▶ Console Output

```
Car HAS-A Engine
Car HAS-A GPS
Car HAS-A MusicSystem
```

Part 04 — Under the Hood: Prototypes, Class Fields & Practical Architecture (24.34 – 24.43)

📌 24.34 — Class vs Object

Module 24.34

| Class | Object | | ----- | ----- | | Blueprint | বাস্তব instance | |
Template | Created entity | | `class Student` | `new Student()` | | অনেক object তৈরি করতে পারে | একটি নির্দিষ্ট
instance | উদাহরণ:

 JavaScript

JAVASCRIPT

```
class Student {}
```

ClassI

 JavaScript

JAVASCRIPT

```
const student = new Student();
```

ObjectI ---

Modern JavaScript-এ `class` বেশি readable। আগে constructor function ব্যবহার করা হতো।

◆ Constructor Function

JavaScript

JAVASCRIPT

```
function Student(name) {
  this.name = name;
}

Student.prototype.introduce = function () {
  console.log(`I am ${this.name}`);
};

const student = new Student("Shariar");

student.introduce();
```

◆ Class Version

JavaScript

JAVASCRIPT

```
class Student {

  constructor(name) {
    this.name = name;
  }

  introduce() {
    console.log(`I am ${this.name}`);
  }

}

const student = new Student("Shariar");

student.introduce();
```

দুটো conceptually related। ---

🚩 24.36 — গুরুত্বপূর্ণ: Class আসলে Prototype-এর উপর কাজ করে

Module 24.36

এটা খুব গুরুত্বপূর্ণ। তুমি Chapter 11-এ Prototype পড়েছো। JavaScript-এর:

JavaScript

JAVASCRIPT

```
class Student {  
  
  introduce() {  
    console.log("Hello");  
  }  
  
}
```

এর methods সাধারণত prototype-এ থাকে। অর্থাৎ:

ASCII Concept Architecture

Class syntax

↓

Prototype-based mechanism

JavaScript মূলত prototype-based language। `class` হলো cleaner syntax/abstraction। ---

🚩 24.37 — Prototype Check

Module 24.37

JavaScript

JAVASCRIPT

```
class Student {  
  
  introduce() {  
    console.log("Hello");  
  }  
  
}  
  
const student = new Student();  
  
console.log(  
  Object.getPrototypeOf(student) === Student.prototype  
);
```

Output:

▶ Console Output

true

Modern JavaScript-এ class-এর মধ্যে directly field declare করা যায়।

JavaScriptJAVASCRIPT

```
class Student {  
  
    name = "Unknown";  
    age = 0;  
  
}  
  
const student = new Student();  
  
console.log(student.name);  
console.log(student.age);
```

Output:

▶ Console Output

```
Unknown  
0
```

JavaScript

JAVASCRIPT

```
class User {  
  
    #password;  
  
    constructor(password) {  
        this.#password = password;  
    }  
  
    checkPassword(password) {  
        return this.#password === password;  
    }  
  
}  
  
const user = new User("12345");  
  
console.log(user.checkPassword("12345"));  
console.log(user.checkPassword("wrong"));
```

Output:

▶ Console Output

```
true  
false
```

Password সরাসরি বাইরে access করা যাবে না। ---

JavaScript

JAVASCRIPT

```
class Product {

  constructor(name, price, stock) {
    this.name = name;
    this.price = price;
    this.stock = stock;
  }

  getTotalPrice(quantity) {

    if (quantity > this.stock) {
      throw new Error("Not enough stock");
    }

    return this.price * quantity;
  }

  sell(quantity) {

    if (quantity > this.stock) {
      throw new Error("Not enough stock");
    }

    this.stock -= quantity;
  }

  showProduct() {
    console.log(
      `${this.name} - ٦${this.price} - Stock: ${this.stock}`
    );
  }
}

const laptop = new Product("Laptop", 60000, 10);

laptop.showProduct();

console.log(
  "Total:",
  laptop.getTotalPrice(2)
);

laptop.sell(2);

laptop.showProduct();
```

Output:

► Console Output

```
Laptop - ₮60000 - Stock: 10  
Total: 120000  
Laptop - ₮60000 - Stock: 8
```

ধরো:

📁 ASCII Concept Architecture

```

Product
  ↓
ElectronicProduct
  ↓
Laptop
    
```

JavaScript

JAVASCRIPT

```

class Product {

  constructor(name, price) {
    this.name = name;
    this.price = price;
  }

  showInfo() {
    console.log(`${this.name}: ₹${this.price}`);
  }
}

class ElectronicProduct extends Product {

  constructor(name, price, warranty) {
    super(name, price);
    this.warranty = warranty;
  }

  showWarranty() {
    console.log(`Warranty: ${this.warranty} months`);
  }
}

const laptop = new ElectronicProduct(
  "Laptop",
  60000,
  24
);

laptop.showInfo();
laptop.showWarranty();
    
```

Output:

► Console Output

Laptop: ₮60000
Warranty: 24 months

একটি বড় project-এ এমন structure দেখা যেতে পারে:

📁 ASCII Concept Architecture

```
src/
├── models/
│   ├── User.js
│   ├── Product.js
│   └── Order.js
├── services/
│   ├── AuthService.js
│   ├── PaymentService.js
│   └── ProductService.js
├── controllers/
│   ├── UserController.js
│   └── ProductController.js
└── app.js
```

উদাহরণ:

JavaScript

JAVASCRIPT

```
class User {

  constructor(name, email) {
    this.name = name;
    this.email = email;
  }

  getProfile() {
    return {
      name: this.name,
      email: this.email
    };
  }
}

export default User;
```

এভাবে application-এর বিভিন্ন responsibility আলাদা করা যায়। ---

ধরো API থেকে user data পেলাম:

JavaScript

JAVASCRIPT

```
const data = {
  id: 1,
  name: "Shariar",
  email: "shariar@example.com"
};
```

Class দিয়ে model:

JavaScript

JAVASCRIPT

```
class User {

  constructor(data) {
    this.id = data.id;
    this.name = data.name;
    this.email = data.email;
  }

  getDisplayName() {
    return this.name;
  }

}

const user = new User(data);

console.log(user.getDisplayName());
```

Output:

▶ Console Output

Shariar

এটি বড় application-এ usefull ---

✦ 24.44 — Common Mistake: new ভুলে যাওয়া

Module 24.44

Class constructor কখনো `new` কীওয়ার্ড ছাড়া সরাসরি ফাংশনের মতো কল করা যায় না। এভাবে কল করলে `TypeError` ছুঁড়ে দেয়।

● ● ● ভুল পদ্ধতি: new কীওয়ার্ড ছাড়া কল

Syntax / Runtime Error

✗ INVALID / ERROR

```
class Student {}  
  
const student = Student();
```

✗ Runtime Error

TypeError: Class constructor Student cannot be invoked without 'new'

সঠিক সমাধান: অবজেক্ট ইনস্ট্যান্স তৈরি করতে সর্বদা `new` ব্যবহার করুন।

● ● ● সঠিক পদ্ধতি: new ব্যবহার

Valid OOP Pattern

✓ CORRECT / VALID

```
class Student {}  
  
const student = new Student();
```

📌 24.45 — Common Mistake: `super()` ভুলে যাওয়া

Module 24.45

Inheritance ব্যবহার করার সময় চাইল্ড ক্লাসের `constructor`-এ `this` অ্যাক্সেস করার পূর্বে অবশ্যই প্যারেন্ট ক্লাসের `super()` কল করতে হবে।

● ● ● ভুল পদ্ধতি: `super()` না ডেকে `this` অ্যাক্সেস

Syntax / Runtime Error

❌ INVALID / ERROR

```
class Animal {
  constructor(name) {
    this.name = name;
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    this.breed = breed;
  }
}
```

❌ Runtime ReferenceError

ReferenceError: Must call super constructor in derived class before accessing 'this'

সঠিক সমাধান: প্যারেন্ট কনস্ট্রাক্টরে প্যারামিটার পাস করতে প্রথমে `super( ... )` কল করুন।

● ● ● সঠিক পদ্ধতি: `super(name)` কল

Valid OOP Pattern

✅ CORRECT / VALID

```
class Animal {
  constructor(name) {
    this.name = name;
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }
}
```

Private field (`#field`) ক্লাসের বাইরে থেকে সরাসরি ডট নোটেসনে অ্যাক্সেস করা যায় না। এটি করলে সিনট্যাক্স এরর দেয়।

● ● ● ভুল পদ্ধতি: ক্লাসের বাইরে Private Field অ্যাক্সেস

Syntax / Runtime Error

✗ INVALID / ERROR

```
class User {
  #password = "12345";
}

const user = new User();
console.log(user.#password);
```

✗ SyntaxError

SyntaxError: Private field '#password' must be declared in an enclosing class

সঠিক সমাধান: ক্লাসের নিজস্ব মেথড বা গেটারের মাধ্যমে প্রাইভেট ডেটা ভ্যালিডেট বা এক্সেস করুন।

● ● ● সঠিক পদ্ধতি: মেথড বা গেটারের মাধ্যমে এক্সেস

Valid OOP Pattern

✓ CORRECT / VALID

```
class User {
  #password = "12345";

  verifyPassword(input) {
    return this.#password === input;
  }
}

const user = new User();
console.log(user.verifyPassword("12345")); // true
```

🚩 24.47 — Common Mistake: Static Method Object দিয়ে Call

Module 24.47

Static method সরাসরি ক্লাসের প্রপার্টি, এটি ইনস্ট্যান্স অবজেক্টে উত্তরাধিকারসূত্রে থাকে না। অবজেক্ট ইনস্ট্যান্স দিয়ে কল করলে `TypeError` দেয়।

● ● ● ভুল পদ্ধতি: অবজেক্ট দিয়ে Static মেথড কল

Syntax / Runtime Error

✗ INVALID / ERROR

```
class Test {
  static hello() {
    console.log("Hello");
  }
}

const test = new Test();
test.hello();
```

✗ TypeError

TypeError: test.hello is not a function

সঠিক সমাধান: সরাসরি ক্লাসের নাম ধরে কল করুন: `Test.hello()`

● ● ● সঠিক পদ্ধতি: ক্লাসের নাম ধরে কল

Valid OOP Pattern

✓ CORRECT / VALID

```
class Test {
  static hello() {
    console.log("Hello");
  }
}

Test.hello(); // Hello
```

🚩 24.48 — Common Mistake: Constructor-এ this

Module 24.48

কনস্ট্রাক্টরের ভেতর `this.property = value` না লিখলে অবজেক্টের প্রপার্টি ইনিশিয়ালাইজ হয় না; শুধু লোকাল ভেরিয়েবল অ্যাসাইন হয়।

● ● ● ভুল পদ্ধতি: this ছাড়া নাম অ্যাসাইন

Syntax / Runtime Error

❌ INVALID / ERROR

```
class Student {
  constructor(name) {
    name = name; // প্যারামিটার নিজের উপর অ্যাসাইন হচ্ছে, ইনস্ট্যান্সে নয়!
  }
}

const s = new Student("Shariar");
console.log(s.name); // undefined
```

► Output Mismatch

undefined

সঠিক সমাধান: অবজেক্টের প্রপার্টি সেট করতে সর্বদা `this.name` ব্যবহার করুন।

● ● ● সঠিক পদ্ধতি: this.name ব্যবহার

Valid OOP Pattern

✅ CORRECT / VALID

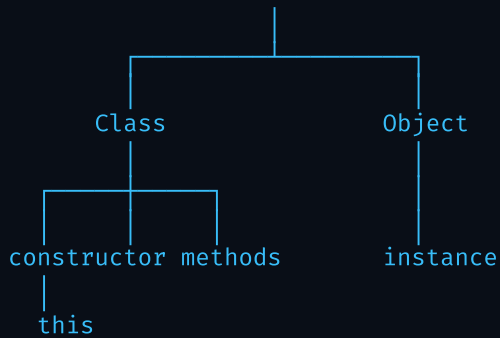
```
class Student {
  constructor(name) {
    this.name = name; // ইনস্ট্যান্স অবজেক্টে প্রপার্টি সেট হলো
  }
}

const s = new Student("Shariar");
console.log(s.name); // 'Shariar'
```

এই পুরো chapter-টা এভাবে মনে রাখো:

🕒 ASCII Concept Architecture

OOP



তারপর:

🕒 ASCII Concept Architecture

Class

- Instance Properties
- Instance Methods
- Static Properties
- Static Methods
- Getters
- Setters
- Private Fields

Inheritance:

🕒 ASCII Concept Architecture

Parent Class

extends



Child Class

super()

OOP pillars:

► Console Output

Encapsulation
Abstraction
Inheritance
Polymorphism

📌 24.50 — MUST KNOW

Module 24.50

এই Chapter থেকে অবশ্যই জানতে হবে:

♦ 1. Class

JavaScript

JAVASCRIPT

```
class Student {}
```

♦ 2. Object

JavaScript

JAVASCRIPT

```
const student = new Student();
```

♦ 3. Constructor

JavaScript

JAVASCRIPT

```
constructor(name) {  
  this.name = name;  
}
```

♦ 4. this

Current instance/object reference

♦ 5. Methods

JavaScript

JAVASCRIPT

```
study() {  
  console.log("Studying");  
}
```

♦ 6. Static

JavaScript

JAVASCRIPT

```
static hello() {}
```

Call:

JavaScript

JAVASCRIPT

```
ClassName.hello();
```

♦ 7. Getter

JavaScript

JAVASCRIPT

```
get name() {}
```

◆ 8. Setter

JavaScript

JAVASCRIPT

```
set name(value) {}
```

◆ 9. Private Field

JavaScript

JAVASCRIPT

```
#password
```

◆ 10. Inheritance

JavaScript

JAVASCRIPT

```
class Dog extends Animal {}
```

◆ 11. Parent constructor

JavaScript

JAVASCRIPT

```
super();
```

◆ 12. Method overriding

Child নিজের method দিয়ে parent method replace/override করতে পারে।

◆ 13. Polymorphism

একই method বিভিন্ন object-এ different behavior দিতে পারে।

◆ 14. Prototype connection

`class` syntax-এর নিচে JavaScript-এর prototype system কাজ করে। ---

JavaScript Class এবং Object-Oriented Programming এর দ্রুত রেফারেন্সের জন্য সম্পূর্ণ সিনটাক্স টেবিল:

Concept	Syntax
Class	<code>class User {}</code>
Constructor	<code>constructor() {}</code>
Object	<code>new User()</code>
Property	<code>this.name</code>
Method	<code>hello() {}</code>
Static	<code>static hello() {}</code>
Getter	<code>get name() {}</code>
Setter	<code>set name(value) {}</code>
Private	<code>#password</code>
Inheritance	<code>extends</code>
Parent constructor	<code>super()</code>
Parent method	<code>super.method()</code>
Instance check	<code>obj instanceof Class</code>

থিওরি এবং কনসেপ্ট আয়ত্ত করার পর বাস্তব প্রজেক্টে ক্লাস ব্যবহারের দক্ষতা যাচাই করার জন্য নিচে ৬টি প্র্যাকটিস প্রবলেমের বিস্তারিত সমাধান দেওয়া হলো:

Practice 1: Car Class — Brand, Model, Year & showInfo() Method

Beginner

Problem Requirement: `Car` নামে একটি ক্লাস বানান যাতে `brand`, `model`, `year` প্রোপার্টি থাকবে এবং `showInfo()` মেথড গাড়ির সম্পূর্ণ বিবরণ প্রিন্ট করবে।

Practice 1 Model Solution

Valid OOP Pattern

✓ CORRECT / VALID

```
class Car {
  constructor(brand, model, year) {
    this.brand = brand;
    this.model = model;
    this.year = year;
  }

  showInfo() {
    console.log(`Car Info: ${this.brand} ${this.model} (${this.year})`);
  }
}

// Verification
const myCar = new Car("Toyota", "Camry", 2024);
myCar.showInfo();
```

► Console Output

```
Car Info: Toyota Camry (2024)
```

Practice 2: Student Class — Name, ID, Department & introduce() Method

Beginner

Problem Requirement: `Student` নামে একটি ক্লাস বানান যাতে `name`, `id`, `department` প্রোপার্টি থাকবে এবং `introduce()` মেথড শিক্ষার্থীর পরিচয় প্রিন্ট করবে।



Practice 2 Model Solution

Valid OOP Pattern



CORRECT / VALID

```
class Student {
    constructor(name, id, department) {
        this.name = name;
        this.id = id;
        this.department = department;
    }

    introduce() {
        console.log(`Hello, I am ${this.name}, ID: ${this.id}, Department of
${this.department}.`);
    }
}

// Verification
const s1 = new Student("Rahim Ahmed", "CSE-1024", "Computer Science");
s1.introduce();
```

► Console Output

Hello, I am Rahim Ahmed, ID: CSE-1024, Department of Computer Science.

Practice 3: Rectangle Class — Width, Height, area() & perimeter() Calculation

Beginner

Problem Requirement: `Rectangle` নামে একটি ক্লাস বানান যার `width` ও `height` প্রোপার্টি থাকবে এবং আয়তক্ষেত্রের ক্ষেত্রফল ও পরিসীমা নির্ণয়ের জন্য `area()` ও `perimeter()` মেথড থাকবে।

● ● ● Practice 3 Model Solution

Valid OOP Pattern

✓ CORRECT / VALID

```
class Rectangle {
  constructor(width, height) {
    this.width = width;
    this.height = height;
  }

  area() {
    return this.width * this.height;
  }

  perimeter() {
    return 2 * (this.width + this.height);
  }
}

// Verification
const rect = new Rectangle(10, 5);
console.log("Area:", rect.area());
console.log("Perimeter:", rect.perimeter());
```

► Console Output

```
Area: 50
Perimeter: 30
```

Practice 4: BankAccount with Private #balance, Deposit & Withdraw Validations Intermediate

Problem Requirement: `BankAccount` ক্লাস বানান যাতে ব্যালেন্স প্রাইভেট (`#balance`) থাকবে এবং `deposit(amount)`, `withdraw(amount)`, ও `getBalance()` মেথডের মাধ্যমে সুরক্ষিতভাবে লেনদেন করা যাবে।

● ● ● Practice 4 Model Solution

Valid OOP Pattern

✓ CORRECT / VALID

```
class BankAccount {
  #balance;

  constructor(initialBalance = 0) {
    this.#balance = initialBalance >= 0 ? initialBalance : 0;
  }

  deposit(amount) {
    if (amount > 0) {
      this.#balance += amount;
      console.log(`Deposited ${amount}. New balance: ${this.#balance}`);
    } else {
      console.log("Deposit amount must be positive.");
    }
  }

  withdraw(amount) {
    if (amount <= 0) {
      console.log("Invalid withdrawal amount.");
    } else if (amount > this.#balance) {
      console.log("Insufficient funds!");
    } else {
      this.#balance -= amount;
      console.log(`Withdrawn ${amount}. Remaining balance:
${this.#balance}`);
    }
  }

  getBalance() {
    return this.#balance;
  }
}

// Verification
const account = new BankAccount(500);
account.deposit(200);
account.withdraw(150);
console.log("Final Balance:", account.getBalance());
// console.log(account.#balance); // ✗ SyntaxError (Fully Protected)
```

► Console Output

```
Deposited $200. New balance: $700
Withdrawn $150. Remaining balance: $550
Final Balance: 550
```

Practice 5: Inheritance Architecture: Employee Base Class to Developer Derived Class

Intermediate

Problem Requirement: `Employee` (name, salary) বেস ক্লাস থেকে `Developer` (language, code()) চাইল্ড ক্লাস তৈরি করুন এবং `super()` এর মাধ্যমে প্যারেন্ট প্রোপার্টি ইনিশিয়ালাইজ করুন।

Practice 5 Model Solution

Valid OOP Pattern

✓ CORRECT / VALID

```
class Employee {
  constructor(name, salary) {
    this.name = name;
    this.salary = salary;
  }

  getDetails() {
    return `${this.name} earns ${this.salary}/year`;
  }
}

class Developer extends Employee {
  constructor(name, salary, language) {
    super(name, salary); // Call parent constructor
    this.language = language;
  }

  code() {
    console.log(`${this.name} is writing code in ${this.language}.`);
  }
}

// Verification
const dev = new Developer("Shariar", 95000, "JavaScript");
console.log(dev.getDetails());
dev.code();
```

► Console Output

```
Shariar earns $95000/year
Shariar is writing code in JavaScript.
```

Practice 6: Mini E-commerce Architecture: Product, Cart, Order & User Interaction Advanced

Problem Requirement: একটি পূর্ণাঙ্গ অবজেক্ট-ওরিয়েন্টেড মিনি ই-কমার্স আর্কিটেকচার তৈরি করুন যাতে `Product`, `Cart`, `Order`, এবং `User` ক্লাসগুলো একে অপরের সাথে ইন্টারঅ্যাক্ট করে মোট মূল্য ও কার্ট প্রসেসিং সম্পন্ন করবে।



Practice 6 Model Solution

Valid OOP Pattern



CORRECT / VALID

```

class Product {
  constructor(id, name, price) {
    this.id = id;
    this.name = name;
    this.price = price;
  }
}

class Cart {
  constructor() {
    this.items = [];
  }

  addProduct(product, quantity = 1) {
    this.items.push({ product, quantity });
    console.log(`Added ${quantity}x ${product.name} to cart.`);
  }

  getTotal() {
    return this.items.reduce((sum, item) => sum + item.product.price *
item.quantity, 0);
  }
}

class Order {
  constructor(user, cart) {
    this.user = user;
    this.items = [...cart.items];
    this.totalAmount = cart.getTotal();
    this.orderDate = new Date();
  }

  printReceipt() {
    console.log(`≡≡≡ RECEIPT FOR ${this.user.name} ≡≡≡`);
    this.items.forEach(i => console.log(`- ${i.product.name} x${i.quantity}:
${i.product.price * i.quantity}`));
    console.log(`Total: ${this.totalAmount}`);
  }
}

class User {
  constructor(name, email) {
    this.name = name;
    this.email = email;
  }
}

// Complete System Simulation
const user = new User("Tanjim", "tanjim@example.com");
const laptop = new Product(101, "MacBook Pro", 1999);
const mouse = new Product(102, "Magic Mouse", 79);

const cart = new Cart();
cart.addProduct(laptop, 1);
cart.addProduct(mouse, 2);

```

```
const order = new Order(user, cart);  
order.printReceipt();
```

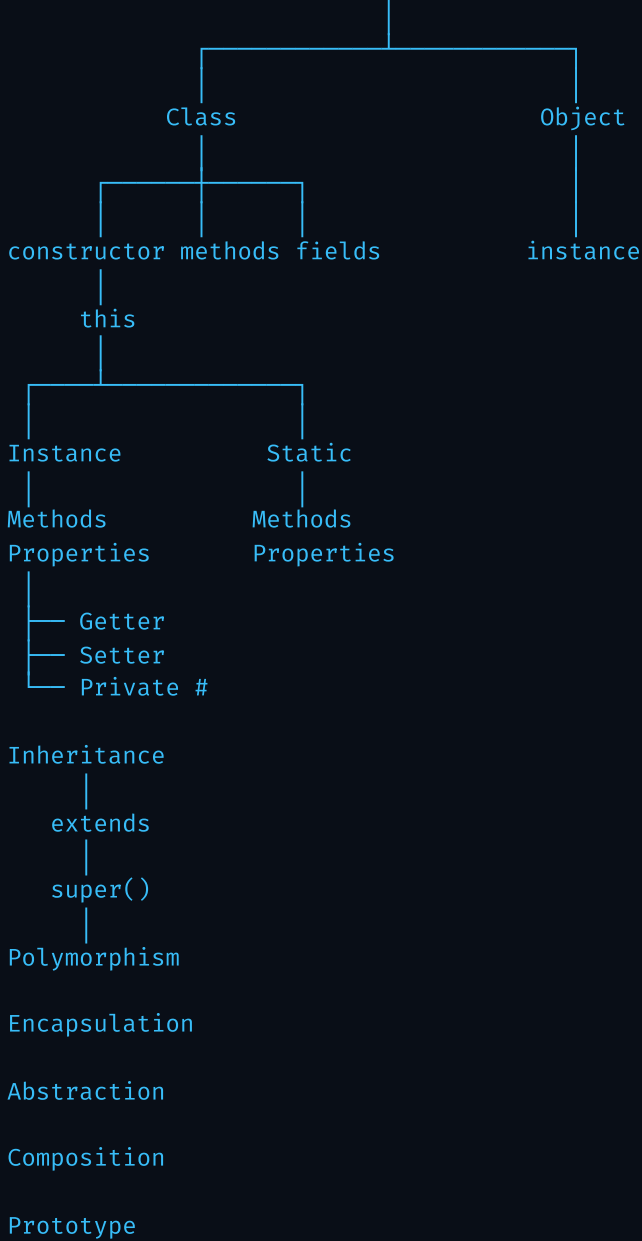
► Console Output

```
Added 1x MacBook Pro to cart.  
Added 2x Magic Mouse to cart.  
≡ RECEIPT FOR Tanjim ≡  
- MacBook Pro x1: $1999  
- Magic Mouse x2: $158  
Total: $2157
```

JavaScript-এর সম্পূর্ণ Object-Oriented Programming ইকোসিস্টেম, ক্লাস ফিল্ডস, ইনহেরিট্যান্স এবং মেথড স্ট্রাকচারের সামগ্রিক ভিজুয়াল আর্কিটেকচার:

JavaScript OOP Master Architecture Tree

JavaScript OOP



★ সবচেয়ে গুরুত্বপূর্ণ Connection

তুমি **Chapter 10-এ Objects**, **Chapter 11-এ Prototypes**, এবং **Chapter 24-এ Classes/OOP** পড়ার পর এই তিনটির পারস্পরিক যোগসূত্র এভাবে মনে রাখবে:

Evolution of JavaScript Object System

Object \n ↓ \n Prototype \n ↓ \n Class syntax \n ↓ \n Inheritance \n ↓ \n OOP architecture

Key Takeaway: JavaScript-এ `class` কোনো সম্পূর্ণ নতুন অবজেক্ট মডেল নয়; এটি আসলে JavaScript-এর বিদ্যমান **Prototypal Inheritance** সিস্টেমের ওপর নির্মিত অত্যন্ত মার্জিত এবং আধুনিক সিনট্যাকটিক সুগার (Syntactic Sugar)। এই কনসেপ্টটি আয়ত্ত করলে React এর ক্লাস কম্পোনেন্ট, TypeScript এর ক্লাস মডেলিং এবং Node.js/NestJS আর্কিটেকচার বোঝা অত্যন্ত সহজ হয়ে যাবে।